

MNC Wildlife Report

Technical Guide

Wildlife sightings and incident reporting across the Mara North Conservancy

Generated August 19, 2026

Workflow id: **mnc-wildlife-report**

1. Overview

The **mnc-wildlife-report** workflow fetches wildlife sighting and incident events from EarthRanger for a specified time window in a single batch and routes them into nine independent reporting branches — one per species group (elephant, buffalo, rhino, lion, leopard, cheetah, giraffe, hartebeest) and one for wildlife incidents (snare, fires, carcasses, injuries, veterinary treatments). Before any event data is fetched, the workflow downloads the MNC conservancy boundary and parcels geospatial files from Dropbox and computes a single shared map view state from the Mara North Conservancy boundary, reused by every map the workflow produces.

The workflow delivers:

- **12 CSV tables** — a raw event dump, cleaned/pivoted wildlife incident records, and per-species sighting summaries
- **13 interactive HTML maps and charts** — herd-composition point maps for elephant and buffalo, clustered herd-size bubble maps and bar charts for elephant and buffalo, category-coloured point maps for lion/leopard/cheetah, plain sighting maps for giraffe/hartebeest/rhino, and an incident-type map
- **1 dashboard** with 21 widgets — one map, chart, or table widget per output above (the two wildlife-incident CSV summaries are not wrapped as dashboard widgets)

Output summary

Output	Type	Branch
wildlife_events.csv	CSV	Raw event dump (all 13 types)
wildlife_incidents_map.html	Map	Wildlife incidents — by type
wildlife_events_recorded.csv	CSV	Wildlife incidents — cleaned
wildlife_incidents_summary_table.csv	CSV	Wildlife incidents — pivoted
wildlife_incidents_recorded_by_date.csv	CSV	Wildlife incidents — by date
elephant_sightings_events.html	Map	Elephant — herd composition
elephant_herd_size_bar_chart.html	Chart	Elephant — herd size bins
elephant_herd_types_map.html	Map	Elephant — herd size bubbles
overall_elephant_summary_table.csv	CSV	Elephant — by herd type
buffalo_sightings_events.html	Map	Buffalo — herd composition
buffalo_herd_size_bar_chart.html	Chart	Buffalo — herd size bins
buffalo_herd_types_map.html	Map	Buffalo — herd size bubbles
overall_buffalo_summary_table.csv	CSV	Buffalo — by herd type
lion_pride_sightings_map.html	Map	Lion — by pride
overall_lion_summary_table.csv	CSV	Lion — by pride
leopard_sightings_map.html	Map	Leopard — by individual
overall_leopard_summary_table.csv	CSV	Leopard — by individual

Output	Type	Branch
cheetah_sightings_map.html	Map	Cheetah — by individual
overall_cheetah_summary_table.csv	CSV	Cheetah — by individual
giraffe_sightings_map.html	Map	Giraffe sightings
overall_giraffe_summary_table.csv	CSV	Giraffe — by date
hartebeest_sightings_map.html	Map	Hartebeest sightings
overall_hart_summary_table.csv	CSV	Hartebeest — by date
rhino_sightings_map.html	Map	Rhino sightings
overall_rhino_summary_table.csv	CSV	Rhino — by date

Note: Earlier versions of this workflow also rendered every map and chart as a PNG via `html_to_png`. The current spec no longer produces PNGs — every map, chart, and summary table is HTML only, persisted with `persist_text` or `persist_df`.

2. Dependencies

2.1 Python packages

Package	Version	Channel
ecoscope-platform	>=2.15.0, <2.16.0	ecoscope-workflows
ecoscope-workflows-ext-custom	0.1.0rc14.*	ecoscope-workflows-custom
ecoscope-workflows-ext-ste	0.0.0rc1.*	ecoscope-workflows-custom
ecoscope-workflows-ext-wwf-virunga	0.0.0rc9.*	ecoscope-workflows-custom
ecoscope-workflows-ext-big-life	1.0.1.*	ecoscope-workflows-custom
ecoscope-workflows-ext-mnc	1.0.1.*	ecoscope-workflows-custom
pydeck	0.9.2	conda-forge
opentelemetry-sdk	>=1.20.0, <2.0.0	conda-forge

Note: **ecoscope-platform** replaces the previously separate **ecoscope-workflows-core** and **ecoscope-workflows-ext-ecoscope** packages. **ecoscope-workflows-ext-wwf-virunga** is a new dependency in this spec, added solely to provide the `draw_bar_chart` task used by the elephant and buffalo herd-size charts. **ecoscope-workflows-ext-mep**, listed in earlier versions of this spec, is no longer required. **opentelemetry-sdk** enables distributed tracing for the compiled workflow's task graph; **pydeck** backs the base-map tile layers.

2.2 Connections and external assets

Asset	Task / Source	Purpose
EarthRanger	<code>set_er_connection</code>	Fetch event records and resolve event detail display titles via <code>process_events_details</code> (used by all branches except giraffe, hartebeest, and rhino need it too — every species branch calls it).
mnc_conservancy.gpkg	<code>ecoscope_workflows_ext_ste.task</code> <code>s.io.fetch_and_persist_file</code> (Dropbox)	MNC community conservancy boundaries. Filtered twice: to <code>grazing_zone == Conservancy</code> for the outline layer, and to <code>name == Mara North Conservancy</code> to compute the shared map view state.
mnc_across_the_river_parcels.gpkg	<code>ecoscope_workflows_ext_ste.task</code> <code>s.io.fetch_and_persist_file</code> (Dropbox)	MNC across-the-river land parcels, rendered as a filled polygon layer on every map.

Note: Both Dropbox files are downloaded with `overwrite_existing: false` and `retries: 3`, via the `ecoscope_workflows_ext_ste.tasks.io` namespace (previously the bare `fetch_and_persist_file` task). If the files already exist from a previous run, the download step is skipped.

2.3 Grouper and global skip conditions

The workflow uses an **empty grouper list** (`groupers: []`). All records are processed as a single undivided dataset; the grouper is passed through to the temporal index and the dashboard only.

A single **task-instance-defaults** block at the top of the spec applies the same skipif conditions — **any_is_empty_df** and **any_dependency_skipped** — to every task automatically, instead of each task repeating an identical skipif block. See Section 8.1 for the one class of task that explicitly overrides this default.

3. Geospatial Asset Pipeline

Before any event data is fetched, the workflow prepares the base map layers and view state shared by every map it produces.

3.1 Base map tile layers

Task: **set_base_maps_pydeck**. Two raster tile layers are configured: an Esri World Hillshade layer (opacity 0.80) and an Esri World Boundaries and Places (Alternate) reference layer (opacity 0.35), both capped at max_zoom 20.

3.2 Conservancy boundary and parcels layers

Step	Task	Detail
1	<code>ecoscope_workflows_ext_ste.tasks.io.fetch_and_persist_file</code>	Download mnc_conservancy.gpkg and mnc_across_the_river_parcels.gpkg from Dropbox.
2	<code>load_df</code>	Load each into a GeoDataFrame (layer: null, deserialize_json: false).
3	<code>ecoscope_workflows_ext_mnc.tasks.transformation.fix_invalid_geometries</code>	Repair invalid geometries in the conservancy boundary GeoDataFrame.
4	<code>filter_df</code> (×2)	Filter the fixed boundary to <code>grazing_zone == 'Conservancy'</code> for the outline layer, and separately to <code>name == 'Mara North Conservancy'</code> for the view-state calculation (Section 3.3).
5	<code>ecoscope_workflows_ext_custom.tasks.results.create_geojson_layer</code> (×2)	Conservancy layer — unfilled, grey outline (<code>get_line_color</code> : [169,169,169], line width 1.75). Parcels layer — filled dark khaki (<code>get_fill_color</code> : [189,183,107,60], line width 1.5, legend title 'Map Layers').

Note: Earlier versions of this workflow split the conservancy boundary into six styled grazing-zone layers (Conservancy Herd Zone, Grazing Zones 1–4) and rendered a text-label layer with conservancy names. Both have been removed from the current spec — every map now uses only the plain conservancy outline and the parcels layer as static background.

3.3 Shared map view state

Task group **Map Zoom & Extent**: `ecoscope_workflows_ext_ste.tasks.spatial_operations.envelope_gdf` computes the bounding envelope of the Mara North Conservancy boundary (`filter_mara_north`), then `ecoscope_workflows_ext_ste.tasks.spatial_operations.compute_view_state_from_gdf` derives a view state from that envelope (pitch: 0, bearing: 0, max_zoom: 15). This single view state (**`gdf_image_extent`**) is passed to every `draw_map` call in the workflow — none of the eleven maps compute their own.

4. Event Ingestion Pipeline

4.1 Event retrieval

Parameter	Value
Task	get_events
event_types	elephant_sighting_rep, buffalo_sighting_rep, rhino_sighting_rep, lion_sighting_rep, leopardsightingrep, cheetah_sighting_rep, giraffe_sighting, hartebeest_sighting, snare_rep, fire_rep, wildlife_injury_rep, wildlife_treatment_rep, wildlife_carcass_rep
Columns retained	id, time, event_type, event_category, reported_by, serial_number, geometry, created_at, event_details, patrols
include_details	true
raise_on_empty	true
include_null_geometry	false
include_updates	false
include_related_events	false
include_display_values	false
force_point_geometry	true

Note: All 13 event types are fetched in a single get_events call, then persisted verbatim as **wildlife_events.csv** before any per-branch filtering. force_point_geometry: true normalises every event's geometry to a point before it reaches the branch pipelines.

4.2 Date extraction and temporal indexing

Step	Task	Detail
1	extract_column_as_type	Extract the time column as output_type: date into a new column named date .
2	add_temporal_index	Add a temporal index using time_col: date , groupers: [], cast_to_datetime: true, format: mixed. Produces events_temporal , the shared input to every branch.

4.3 Branch filters

Every branch begins by filtering events_temporal to its own event type(s). Species branches use **filter_df** (op: equal); the wildlife incidents branch uses **filter_row_values** to capture five event types in one step.

Branch	Filter task	event_type value(s)
Wildlife incidents	filter_row_values	snare_rep, fire_rep, wildlife_injury_rep, wildlife_treatment_rep, wildlife_carcass_rep

Branch	Filter task	event_type value(s)
Elephant	filter_df	elephant_sighting_rep
Buffalo	filter_df	buffalo_sighting_rep
Lion	filter_df	lion_sighting_rep
Leopard	filter_df	leopardsightingrep
Cheetah	filter_df	cheetah_sighting_rep
Giraffe	filter_df	giraffe_sighting
Hartebeest	filter_df	hartebeest_sighting
Rhino	filter_df	rhino_sighting_rep

4.4 Common species normalisation pattern

Every species branch runs the same three steps after filtering, using the EarthRanger client to resolve display titles:

Step	Task	Detail
1	process_events_details	Resolve event detail field IDs to display titles (map_to_titles: true, ordered: true).
2	normalize_json_column	Flatten the event_details JSON column (skip_if_not_exists: true, sort_columns: true).
3	drop_column_prefix	Remove the event_details__ prefix from all flattened columns (duplicate_strategy: keep_original).

Elephant, buffalo, lion, leopard, and cheetah additionally run a **map_columns** step to drop housekeeping columns and rename fields to snake_case (parameters vary per branch — see Sections 6–9). Giraffe, hartebeest, and rhino skip map_columns entirely and pass the prefix-dropped output directly into their map and summary steps.

5. Branch 1 — Wildlife Incidents

Captures five incident event types in one filter_row_values step and produces three CSV tables plus a point map.

5.1 Normalisation and column selection

Step	Task	Detail
1	process_events_details	Resolve field IDs to display titles (map_to_titles: true, ordered: true).
2	normalize_json_column	Flatten event_details (skip_if_not_exists: true, sort_columns: true).
3	drop_column_prefix	Remove the event_details__ prefix (duplicate_strategy: keep_original).
4	map_columns	Drop index, time, event_category, reported_by, serial_number (retain_columns: [], rename_columns: {}, raise_if_not_found: false).

5.2 Outputs

Step	Task	Output
1	persist_df	wildlife_events_recorded.csv — cleaned incident records.
2	make_wildlife_summary_table	ecoscope_workflows_ext_mnc custom task. Pivots events by type with readable labels (fire_rep→Fire, snare_rep→Snare, wildlife_carcass_rep→Wildlife carcass, wildlife_injury_rep→Injured wildlife, wildlife_treatment_rep→Veterinary treatment; max_unique: 6, shorten_width: 300).
3	persist_df	wildlife_incidents_summary_table.csv — pivoted incident summary.
4	summarize_df	Group by date ; nunique id → no_of_events . reset_index: true.
5	persist_df	wildlife_incidents_recorded_by_date.csv — daily incident count (no totals row is appended in the current spec).

5.3 Incident map

Step	Task	Detail
1	ecoscope_workflows_ext_custom.tasks.transformation.drop_null_geometry	Drop rows with null geometry.
2	apply_color_map	Colour points by event_type using tab10 → column colors .
3	map_column_value	Map event_type → event_type_mapped with readable labels (default: Undefined, keep_unmapped: false).
4	ecoscope_workflows_ext_custom.tasks.results.create_scatterplot_layer	Render points: get_radius: 3, opacity: 0.55. Legend title: 'Wildlife Incidents', sorted ascending.
5	ecoscope_workflows_ext_ste.tasks.spatial_operations.combine_deckgl_map_layers	Static layers: parcels + conservancy boundary.

Step	Task	Detail
6	ecoscope_workflows_ext_big_life.tasks.results.draw_map	max_zoom: 10, legend placement: bottom-right, view_state: shared gdf_image_extent.
7	persist_text	Save as wildlife_incidents_map.html .
8	create_map_widget_single_view	“Wildlife Incident Map” dashboard widget (skipif: [never] — always runs, see Section 8.1).

6. Branch 2 — Elephant Sightings

Filters **elephant_sighting_rep** events and produces a herd-composition point map, a herd-size bar chart, a herd-size bubble map, and a herd-type summary table.

6.1 Column mapping

Task: **map_columns** (raise_if_not_found: false). Dropped: index, time, event_type, event_category, reported_by, serial_number, event_type_display, Comments. Renamed:

Source column (title after prefix drop)	Renamed to
Female	female
Herd Demographic	herd_composition
Herd size	herd_size
Male	male
Subadult	sub_adult
< 1 year	underayear

6.2 Cleaning and value mapping

Step	Task	Detail
1	fill_missing_values	Fill nulls in herd_composition with Undefined .
2	fill_missing_values	Fill nulls in underayear, female, male, herd_size, sub_adult with 0.
3	convert_columns_to_int	Cast underayear, female, male, herd_size, sub_adult to integer (errors: coerce, fill_value: 0).
4	map_column_value	Standardise herd_composition → herd_composition_mapped: bachelor→Bachelor, femalecalf→Female + Calf, mixed→Mixed, Undefined/Unspecified→Unspecified (default: Undefined, keep_unmapped: false).

6.3 Herd-size bar chart

Step	Task	Detail
1	apply_classification	Bin herd_size into 5 equal-interval bins (scheme: equal_interval, k: 5, label_ranges: true) → column herd_size_bins .
2	order_categorical_by_number	Order herd_size_bins as a sortable category.
3	ecoscope_workflows_ext_wf_virunga.tasks.plot.draw_bar_chart	Bar chart: category = herd_size_bins; count of id; marker_color: #6495ed; axis titles 'Group size' / 'Number of records'; bargap/bargroupgap: 0.05.

Step	Task	Detail
4	<code>persist_text</code>	Save as elephant_herd_size_bar_chart.html .
5	<code>create_plot_widget_single_view</code>	"Elephant Herd Size Distribution" dashboard widget.

6.4 Herd-size bubble map

Step	Task	Detail
1	<code>ecoscope_workflows_ext_custom.tasks.transformation.drop_null_geometry</code>	Drop rows with null geometry from the ordered/binning table.
2	<code>ecoscope_workflows_ext_big_life.tasks.results.create_clustered_labeled_scatterplot_layer</code>	Clustered bubble layer: <code>get_value/get_radius = herd_size</code> , <code>aggregate: true</code> , <code>cluster_max_radius: 40</code> , fixed colour [0,191,255] (deep sky blue), white text labels.
3	<code>combine_deckgl_map_layers + draw_map</code>	Static layers: parcels + conservancy boundary. Legend title: 'Herd Size'.
4	<code>persist_text</code>	Save as elephant_herd_types_map.html .
5	<code>create_map_widget_single_view</code>	"Elephant Herd Size Map" dashboard widget.

6.5 Herd-composition point map

Step	Task	Detail
1	<code>apply_color_map</code>	Colour by herd_composition using tab10 → column <code>herd_composition_colors</code> .
2	<code>create_scatterplot_layer</code>	<code>get_radius: 3</code> , <code>opacity: 0.55</code> . Legend title: 'Herd Types'.
3	<code>combine_deckgl_map_layers + draw_map</code>	Static layers: parcels + conservancy boundary.
4	<code>persist_text</code>	Save as elephant_sightings_events.html .
5	<code>create_map_widget_single_view</code>	"Elephant Herd Composition Map" dashboard widget.

6.6 Herd-type summary table

Task: **summarize_df** grouped by **herd_composition_mapped**, `nunique id` → observations. Renamed to 'Herd Type' via `map_columns`, persisted as **overall_elephant_summary_table.csv**, rendered with `draw_table` (sorting/filtering enabled, download disabled), and wrapped as the "Elephant Herd Composition Summary" table widget.

7. Branch 3 — Buffalo Sightings

Filters **buffalo_sighting_rep** events. The pipeline mirrors the elephant branch exactly (Section 6) — column mapping, cleaning, herd-size binning, bar chart, bubble map, composition map, and summary table — with one structural difference: buffalo events carry no female/male/sub-adult/under-a-year demographic breakdown, only **herd_composition** and **herd_size**.

7.1 Column mapping

Source column	Renamed to
Herd Demographic	herd_composition
Herd Size	herd_size

Note: Dropped columns: index, time, event_type, event_category, reported_by, serial_number, event_type_display, Comment (singular — 'Comment', not 'Comments' as in the elephant branch).

7.2 Outputs

Output	Description
buffalo_sightings_events.html	Buffalo locations coloured by herd_composition (tab10)
buffalo_herd_size_bar_chart.html	Bar chart of buffalo herd size bins (equal_interval, k=5)
buffalo_herd_types_map.html	Clustered bubble map of buffalo herd sizes
overall_buffalo_summary_table.csv	Sighting counts by herd type

8. Branch 4 — Lion Sightings

Filters **lion_sighting_rep** events and produces a pride-coloured point map and a pride summary table.

8.1 Column mapping

Task: **map_columns** (raise_if_not_found: false). Dropped: index, time, event_type, event_category, reported_by, serial_number, event_type_display, Comment, Behavior. Renamed:

Source column	Renamed to
Female	female
Male	male
Group Size	group_size
Individuals Present	individuals_present
Pride	pride
Young	young

8.2 Cleaning and value mapping

Step	Task	Detail
1	fill_missing_values	Fill nulls in pride with Undefined .
2	map_column_value	Map pride → mapped_pride: Undefined→Undefined, 'Other (specify in comments)'→Undefined, Unknown→Undefined (default: Undefined, keep_unmapped: true — named prides pass through unchanged).
3	apply_color_map	Colour by mapped_pride using tab10 → column colors.

8.3 Outputs

Output	Description
lion_pride_sightings_map.html	Lion locations coloured by mapped_pride (tab10)
overall_lion_summary_table.csv	Sighting counts grouped by pride, renamed to 'Pride'

9. Branch 5 — Leopard Sightings

Filters **leopardsightingrep** events (note: no underscore between 'leopard' and 'sighting' in the raw event type string). Structurally identical to the lion branch, but keyed on individuals present rather than pride.

9.1 Column mapping

Task: **map_columns** (raise_if_not_found: false). Dropped: index, time, event_type, event_category, reported_by, serial_number, event_type_display, Comment, Behavior. Renamed: Female→female, Male→male, Group Size→group_size, Individuals Present→individuals_present, Young→young. (No Pride field for leopard.)

9.2 Cleaning and value mapping

Step	Task	Detail
1	fill_missing_values	Fill nulls in individuals_present with Undefined .
2	map_column_value	Map individuals_present → individuals_present_mapped: Undefined→Undefined, 'Other (specify in comments)'→Undefined, Unknown→Undefined (keep_unmapped: true).
3	apply_color_map	Colour by individuals_present_mapped using tab10 .

9.3 Outputs

Output	Description
leopard_sightings_map.html	Leopard locations coloured by individuals_present_mapped. Legend title: 'Individuals'.
overall_leopard_summary_table.csv	Sighting counts grouped by individuals_present, renamed to 'Individuals'

10. Branch 6 — Cheetah Sightings

Filters **cheetah_sighting_rep** events. The pipeline is identical to the leopard branch (Section 9) — same column mapping, same cleaning, same value mapping on **individuals_present**.

10.1 Outputs

Output	Description
cheetah_sightings_map.html	Cheetah locations coloured by individuals_present_mapped. Legend title: 'Individuals'.
overall_cheetah_summary_table.csv	Sighting counts grouped by individuals_present, renamed to 'Individuals'

11. Branches 7–9 — Giraffe, Hartebeest, Rhino

These three branches are the simplest in the workflow: after the common normalisation pattern (Section 4.4) there is no `map_columns` step, no value mapping, and no herd-size logic — just a fixed-colour point map and a daily-count summary table.

11.1 Filters and map colour

Branch	event_type value	Map file	Point colour
Giraffe	giraffe_sighting	giraffe_sightings_map.html	[72, 61, 139] dark slate blue
Hartebeest	hartebeest_sighting	hartebeest_sightings_map.html	[72, 61, 139] dark slate blue
Rhino	rhino_sighting_rep	rhino_sightings_map.html	[72, 61, 139] dark slate blue

Each map: **drop_null_geometry** → **create_scatterplot_layer** (get_radius: 3, opacity: 0.55, single fixed legend entry 'Sighting') → **combine_deckgl_map_layers** (parcels + conservancy boundary) → **draw_map** → **persist_text** → **create_map_widget_single_view**.

11.2 Daily-count summary tables

Branch	Output CSV	Widget
Giraffe	overall_giraffe_summary_table.csv	Giraffe Sightings Summary
Hartebeest	overall_hart_summary_table.csv	Hartebeest Sightings Summary
Rhino	overall_rhino_summary_table.csv	Rhino Sightings Summary

Note: Each table is produced by `summarize_df` grouped by **date** (unique id → observations), persisted as CSV, rendered with `draw_table`, and wrapped as a table widget — the same pattern used for the herd-type summaries in Sections 6 and 7, but grouped by date instead of a category column. No totals row is appended.

12. Output Files

All outputs are written to **ECOSCOPE_WORKFLOWS_RESULTS**.

12.1 CSV tables

File	Branch	Description
wildlife_events.csv	All	Raw dump of all 13 fetched event types, pre-branching
wildlife_events_recorded.csv	Wildlife incidents	Cleaned incident records
wildlife_incidents_summary_table.csv	Wildlife incidents	Pivoted incident summary by type
wildlife_incidents_recorded_by_date.csv	Wildlife incidents	Daily unique incident count
overall_elephant_summary_table.csv	Elephant	Sighting counts by herd type
overall_buffalo_summary_table.csv	Buffalo	Sighting counts by herd type
overall_lion_summary_table.csv	Lion	Sighting counts by pride
overall_leopard_summary_table.csv	Leopard	Sighting counts by individuals present
overall_cheetah_summary_table.csv	Cheetah	Sighting counts by individuals present
overall_giraffe_summary_table.csv	Giraffe	Daily unique sighting count
overall_hart_summary_table.csv	Hartebeest	Daily unique sighting count
overall_rhino_summary_table.csv	Rhino	Daily unique sighting count

12.2 Maps and charts

File	Branch	Coloured by
wildlife_incidents_map.html	Wildlife incidents	event_type_mapped
elephant_sightings_events.html	Elephant	herd_composition
elephant_herd_size_bar_chart.html	Elephant	herd_size_bins (bar chart)
elephant_herd_types_map.html	Elephant	fixed colour, bubble size = herd_size
buffalo_sightings_events.html	Buffalo	herd_composition
buffalo_herd_size_bar_chart.html	Buffalo	herd_size_bins (bar chart)
buffalo_herd_types_map.html	Buffalo	fixed colour, bubble size = herd_size
lion_pride_sightings_map.html	Lion	mapped_pride
leopard_sightings_map.html	Leopard	individuals_present_mapped

File	Branch	Coloured by
cheetah_sightings_map.html	Cheetah	individuals_present_mapped
giraffe_sightings_map.html	Giraffe	fixed colour
hartebeest_sightings_map.html	Hartebeest	fixed colour
rhino_sightings_map.html	Rhino	fixed colour

13. Workflow Execution Logic

13.1 Global skip conditions and widget-creation override

This workflow defines a single **task-instance-defaults** block applying **any_is_empty_df** and **any_dependency_skipped** as skipif conditions to every task by default. Every **create_map_widget_single_view**, **create_plot_widget_single_view**, and **create_table_widget_single_view** task explicitly overrides this with **skipif: conditions: [never]** — these widget-wrapper tasks always run, regardless of whether their upstream map, chart, or table was skipped.

Note: This means every one of the 21 entries in the dashboard's widgets list is always populated, even for a branch whose underlying event data was empty for the run — the widget-creation task itself never skips, only the data-producing tasks feeding it do.

13.2 Nine independent branches

Branch	Event type(s)	CSV outputs	HTML outputs
Wildlife incidents	snare_rep, fire_rep, wildlife_injury_rep, wildlife_treatment_rep, wildlife_carcass_rep	3	1 map
Elephant	elephant_sighting_rep	1	3 (map + bar chart + bubble map)
Buffalo	buffalo_sighting_rep	1	3 (map + bar chart + bubble map)
Lion	lion_sighting_rep	1	1 map
Leopard	leopardsightingrep	1	1 map
Cheetah	cheetah_sighting_rep	1	1 map
Giraffe	giraffe_sighting	1	1 map
Hartebeest	hartebeest_sighting	1	1 map
Rhino	rhino_sighting_rep	1	1 map

13.3 No PNG rendering

Unlike earlier versions of this workflow, the current spec contains no **html_to_png** steps anywhere. Every map, bar chart, and summary table is persisted as HTML only (via **persist_text** or **persist_df** + **draw_table** + **persist_text**), and consumed directly by the dashboard widgets — nothing is rasterised to a static image.

13.4 Dashboard

The workflow concludes with **gather_dashboard** (step name “MNC event report dashboard”, id: **mnc_events_dashboard**), which packages workflow details, time range, groupers, and a **widgets** list of all **21** map/chart/table widgets produced across the nine branches — one map widget per branch, plus a chart widget and table widget for elephant/buffalo, and a table widget for every other species branch. The wildlife incidents branch is the only one whose CSV summaries (pivoted-by-type and by-date) are not wrapped as table widgets.

14. Software Versions

Package	Version pinned in spec.yml
ecoscope-platform	>=2.15.0, <2.16.0
ecoscope-workflows-ext-custom	0.1.0rc14.*
ecoscope-workflows-ext-ste	0.0.0rc1.*
ecoscope-workflows-ext-wwf-virunga	0.0.0rc9.*
ecoscope-workflows-ext-big-life	1.0.1.*
ecoscope-workflows-ext-mnc	1.0.1.*
pydeck	0.9.2
opentelemetry-sdk	>=1.20.0, <2.0.0